

SIGSPATIAL '26 Presentation Speaker Script

Slide 1 — Title

~35-45 seconds

Good morning everyone.

My name is Abdallah Abdelwahab, from the University of Alabama.

Today, I'm presenting our work titled "**Lexicographic Multi-Preference Routing for Priority-Aware Urban Navigation.**"

This work started from a very simple question:

Why should a navigation system decide what matters most to me?

Current navigation systems are very good at finding fast routes. But in many situations, travel time is not the only thing a user cares about.

A user may care more about safety, congestion, traffic signals, or even following roads that people commonly use.

And more importantly, different users may prioritize these criteria differently.

So our goal is to give the user control over **what matters first**.

Transition:

Let me start with the problem.

Slide 2 — The Problem

~55-65 seconds

Today, when we use a navigation application, the routing objective is usually centered around travel time.

We may get a few switches such as:

avoid tolls,
avoid highways,
or avoid ferries.

But suppose I say:

"I want the safest route."

Or:

"I care about safety first, but if two routes are similarly safe, then I prefer the faster one."

Current systems do not really give us this level of control.

And this matters because different users have different priorities.

For example, a parent driving at night may care about safety above everything else.

A delivery driver may care mainly about congestion.

Another user may want to avoid routes with many traffic signals.

So the problem is not simply that users have multiple preferences.

The important part is that **those preferences have an order.**

Transition:

And that observation leads directly to the main idea of our work.

Slide 3 — Our Insight

~45-55 seconds

People usually do not think in numerical weights.

A normal user probably does not say:

"Give safety a weight of 0.63, traffic 0.24, and popularity 0.13."

Instead, people naturally say:

"Safety first, then the usual way."

Or:

"Fastest first, then safety."

This is a **priority order.**

So instead of asking users to choose weights, we let them rank their preferences.

And we treat this ordering as a contract:

a lower-priority preference should not override a higher-priority one.

This is the motivation for using **lexicographic optimization**.

Transition:

But supporting this idea in a real navigation system creates two challenges.

Slide 4 — Two Obstacles

~60 seconds

The first challenge is the data.

Safety, historical popularity, congestion, and intersection delays do not come directly from the road graph.

They come from completely different sources.

Crash records have one format.

Historical trajectories have another.

Traffic information comes from a live service.

And traffic signals and other intersection attributes come from OpenStreetMap.

So first, we need to convert all these heterogeneous sources into something the routing algorithm can actually use.

The second challenge is optimization.

Even after we have all these criteria, how do we combine them while respecting the user's priority order?

A weighted sum mixes everything into one number.

A Pareto approach may return many alternatives and leave the final choice to the user.

We wanted something different:

one route, respecting the stated priority order, and fast enough for interactive navigation.

Transition:

This is the system we built to address both challenges.

Slide 5 — System Overview

~70-80 seconds

This figure summarizes the complete framework.

There are two main parts: **offline processing** and **online routing**.

On the left, we integrate several urban data sources.

We use OpenStreetMap to construct the road graph and obtain road and intersection attributes.

We use NYC crash records to model crash exposure.

We use historical routes to capture which road segments are commonly used toward a destination.

And for real-time traffic, we can obtain current speeds from the TomTom Traffic Flow API.

Offline, these data sources are converted into preference-specific scores attached to the road network.

We also divide the map into tiles and precompute heuristic information.

Then, online, the user provides an origin, destination, and an ordered set of preferences.

For example:

safety, then traffic, then historical popularity.

Lexicographic FreqA* uses those scores and the precomputed heuristic to return one route.

The important point is that changing the preference order does **not** require rebuilding the entire system.

The same precomputed structures can be reused.

Transition:

So the next question is: how do we represent these very different preferences in the same routing framework?

Slide 6 — Preferences on a Common Scale

~70-80 seconds

Here we visualize three preference fields over Manhattan.

The first is **crash exposure**.

You can see that crashes are concentrated more strongly along certain corridors.

The second is **historical usage**.

Here, the stronger areas correspond to road segments that appear more frequently in historical routes.

And the third field represents the traffic preference, including road-speed information and penalties associated with controlled intersections.

Although these data come from different sources, we convert each preference into scores between zero and one.

For a particular preference, a route score is obtained by multiplying its transition scores along the path.

And each transition can include both an **edge factor** and a **node factor**.

So, for safety, for example, we can penalize both a road segment with crash exposure and an intersection with recorded crashes.

For traffic, we can favor faster road segments while also penalizing intersections such as traffic signals.

The important visual point is that these fields are different.

So when we prioritize a different preference, the search is pushed toward a genuinely different part of the road network.

Transition:

Once we have these preference scores, we still need to search efficiently. That brings us to the main algorithm.

Slide 7 — Lexicographic FreqA*

~90 seconds

Our method has two main ideas.

The first is **lexicographic search**.

Instead of giving each route one combined score, each route has a **score vector**.

For example:

safety score,
traffic score,
and historical score.

We compare candidates using the user's priority order.

So if safety is first, we compare safety first.

Only when two candidates are sufficiently close on safety do we allow the next preference to influence the decision.

We control this using the tolerance epsilon.

At **epsilon equal to zero**, the ordering is strictly lexicographic and we have the formal optimality guarantee.

In the experiments, we use a small tolerance, epsilon equal to 0.05, so that a secondary preference can influence near-ties without substantially changing the primary objective.

The second idea is the **tile-graph heuristic**.

Instead of searching the road graph blindly, we create a coarse abstraction of the network using tiles.

For each preference, the tile graph gives an optimistic estimate of the best score that can still be achieved toward the destination.

Because this heuristic is admissible and consistent, it can guide A* while preserving the theoretical properties of the search.

And because the heuristic is precomputed, we can reuse it when the user changes the preference order.

Transition:

The theory is useful, but the most important question is whether changing the order actually changes the route.

Slide 8 — Result 1: Safety and Historical Preference

~45-55 seconds

Here we look at the same origin and destination while changing what the user prioritizes.

On these examples, when we prioritize safety, the route moves away from areas with greater crash exposure.

When historical popularity is prioritized, the route instead follows the segments that appear more strongly in the historical usage field.

The origin and destination did not change.

The road network did not change.

The only thing we changed was the user's preference order.

And yet the resulting paths are visibly different.

So priority ordering is not simply changing a score attached to the same route.

It is changing the route itself.

Transition:

We observe the same behavior when traffic becomes the priority.

Slide 9 — Result 1: Traffic Preferences

~40–50 seconds

Here we continue with the same idea.

The traffic-oriented routes respond to a different field.

The offline traffic preference considers the baseline road and intersection information.

The online traffic preference can instead use current traffic conditions.

Across the offline preference-first routes, the paths are almost edge-disjoint, with pairwise overlap of at most about **2.6 percent** for the illustrated query.

That is an important result for us.

It shows that the preference order is not a cosmetic reweighting.

It materially reshapes the selected path.

Transition:

But preference-aware routing is only useful if we can compute these routes quickly.

Slide 10 — Result 2: Fast and Optimal

~75–90 seconds

This experiment evaluates search efficiency.

Our main baseline here is FreqDijkstra, which optimizes the same historical-frequency objective but without our tile heuristic.

FreqDijkstra visits around **1,249 nodes** on average and takes approximately **41 milliseconds**.

FreqA*, using our tile heuristic, visits about **250 nodes** and takes only **0.42 milliseconds**.

So we reduce node expansions by approximately **80 percent**, while still achieving **100 percent FreqScore**.

In other words, for this single-preference experiment, we reach exactly the same optimal historical-frequency route while exploring much less of the graph.

That corresponds to approximately a **97.8-times speedup** over FreqDijkstra.

Our multi-preference Lex-FreqA* remains at millisecond-scale latency as well, with about **1.13 milliseconds** in this experiment.

So the heuristic is not just theoretically valid.

It gives us a substantial practical reduction in search effort.

Transition:

So far, these experiments are mostly based on static information. But traffic is not static.

Slide 11 — Result 3: Live Traffic

~60–70 seconds

Here we ask exactly the same traffic-first query at two different times.

The first query is at 7:30.

The second is at 11:42.

Between those times, the origin and destination remain the same.

The user's preference remains the same.

But the traffic conditions change.

And as you can see, the selected route changes as well.

The algorithm favors free-flowing segments and avoids corridors that are congested at the time of the query.

This is important because we are not returning a cached traffic route.

The search is evaluated against the current traffic state.

And the routing search itself remains fast, at around **2.2 milliseconds** in this experiment.

Transition:

Of course, respecting a strong preference can have a cost.

Slide 12 — Result 4: The Cost of Preference

~70-80 seconds

If I tell the system that safety is more important than distance, I should not expect the result to always be the shortest route.

And that is exactly what we observe.

The historical-first route is about **20 percent longer** than the shortest path on average.

Safety-first is around **60 percent longer**.

And traffic-first is around **63 percent longer**.

We do not interpret these detours simply as inefficiency.

They are the measurable cost of honoring the preference.

But users may not accept an unlimited detour.

So we also introduce an optional **distance budget**.

For example, with beta equal to 2, we allow a route to be at most twice the shortest-path distance.

In the safety experiment, this reduces the detour substantially while retaining about **85 percent** of the unconstrained safety quality.

So the system gives the user another practical control:

respect my preference, but do not send me arbitrarily far away.

Transition:

Before I conclude, I want to be clear about what these experiments do—and do not—demonstrate.

Slide 13 — Scope and Limitations

~65-75 seconds

There are several limitations that are important to state clearly.

First, the historical-route dataset in this study is synthetic.

So when we say the historical route is optimal, we mean optimal with respect to the frequency field used by the system.

We are not claiming that this perfectly represents real driver behavior.

A real trajectory dataset can replace this frequency table without changing the search algorithm itself.

Second, our safety measure uses raw crash exposure.

We do not normalize crashes by traffic volume.

So a busy road can accumulate more crashes simply because more vehicles use it.

That means the safety detour reported here may be an upper bound.

And finally, the evaluation is currently focused on Manhattan.

These are limitations of the preference fields and evaluation scope.

They do not change the underlying search guarantees or the efficiency of the algorithm for a given preference field.

Transition:

So let me finish with the main idea I would like you to remember.

Slide 14 — Takeaway

~45-55 seconds

The main message of this work is simple.

Users should be able to tell a navigation system:

“Give me the safest route first, and then, among similar choices, give me the fastest one.”

They should not need to invent numerical weights.

And they should not need to examine a large set of Pareto-optimal alternatives.

Our framework turns multiple urban data sources into preference-aware routing criteria, lets the user explicitly specify their priority order, and uses Lexicographic FreqA* to find a single priority-respecting route efficiently.

The same framework can also adapt when live traffic changes.

So rather than the navigation system deciding what the user values,

we let the user decide what matters first.

Thank you very much.

I'll be happy to take your questions.